AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Summer 2024-2025 Section: [*F*]

Group No: 04

Project Report On

Project Name: Pharmacy Management System

Supervised By
Md. Hasibul Hasan

Submitted By:

Name	ID
1. Sourovi Akter Purnima	23-55016-3
2. Sadia Khandaker Moumita	23-52278-2
3. Meherin Sikder	23-52578-2

Obtained Marks for CO2 and CO3 (Description given in the following page)

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)

Evaluation Criteria (CO2)	Total =	Evaluation Criteria (CO3)	Total =
Requirement fulfillment		Organization of the application	
Validation		Representation and Integration of Database	
Verification		Graphical User Interface	

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	**Evaluation Definition**				
Requirement fulfillment	Fails to demonstrate any understanding of real-life scenariobased project development or functional requirement identification. There is no attempt to depict a project or identify functional requirements accurately.	Demonstrates limited understanding of real-life scenariobased project development and functional requirement identification. The project depicted lacks coherence or relevance to reallife scenarios, and functional requirements are inaccurately identified or insufficiently described.	Presents a basic depiction of a real-life scenariobased project and identifies some functional requirements. However, the project lacks depth or complexity, and some functional requirements may be vaguely defined or missing key details.	Effectively demonstrates a realistic scenariobased project and accurately identifies most functional requirements. The project is welldeveloped with appropriate complexity, and functional requirements are clearly articulated with relevant details.	Exhibits an exceptional understanding of real-life scenariobased project development and accurately identifies all functional requirements. The project is meticulously developed with thorough attention to detail, reflecting a comprehensive understanding of Object-Oriented Programming project development activities.
Validation	Fails to demonstrate any understanding or implementation of validation forms in their system. There is no attempt to deal with data validation, and validation requirements are completely ignored or incorrectly applied.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation.	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation.	Effectively demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system.	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall reliability and integrity of the system.

Verification	Fails to demonstrate any attempt to verify the system data or functional requirements. There is no evidence of understanding or implementation	Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts are incomplete or	Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may be inconsistent or	Identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and thorough, with attention to	Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive
	of verification processes, and data flow is not considered.	inaccurate, and there is insufficient consideration given to ensuring data integrity and functionality.	lack thoroughness, and there may be gaps in ensuring proper functional requirements and data flow.	ensuring data integrity and appropriate data flow within the system.	and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.

CO3: Prepare and Explain a real life desktop based application synthesizing several component of C# along with development tools to adhere the given requirements.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	**Evaluation Definition**				
Organization of the application	Fails to identify any suitable real time application or requirements for project development activities related to OOP.	Limited understanding about the project scopes and scenarios or identification of functional requirements.	Lacks depth or relevance to OOP project development activities and may contain inaccuracies. Real-life scenarios are mentioned, but the discussion lacks depth or clarity.	Consider and integrate the idea of several core aspects of the project along with relevance to real-life scenarios. Demonstrating a solid understanding of the application presentation.	Generalize and exhibits an exceptional understanding of project preparation according to a to real-life scenarios. Also contains proper and insightful identification of the system which is comprehensive and precise.

Representation and Integration of Database	Fails to identify and present any understanding or implementation of database. Also failed to integrate the data with the project itself.	Limited understanding of the database concepts or their proper way of using in a real time project. While some attempt may be made to implement but it is incomplete or poorly executed, leading to inadequate design.	Lacks depth or relevance to database integration with the application. Shows a basic understanding but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistency. May lack proper normalization.	Integrate the database with the forms properly and implements it with proper validation which is mostly accurate and comprehensive, ensuring the proper handling of data input and verification along with general normalization.	Exhibits an exceptional understanding and implementation of database ensuring attention to detail, and robust data manipulation procedures and contributing to the overall clarity.
Graphical User Interface	Fails to present or prepare GUI based application interfaces. There is no evidence of creating or integrating such things according to their usefulness.	Limited understanding of graphical user interfaces. Lack of design knowledge. Very poor attempt to make such things which are currently obsolete or can't be identified as coherent.	Shows a basic understanding of creating user interfaces. Most of them are interconnected but maybe some of them lack it. However, most of it can be described as user friendly.	Effectively identifies and meet the consider the simplicity. Design related works are mostly accurate and taken proper attention to ensuring a userfriendly coherent system.	Exhibits an exceptional work design following a high standard of simple and elegant work. Several controls and mechanism has been organized in a preferred way according to the coherent usage .

Table of Contents: **Page no.**

1. Chapter: 01 (Introduction)---------- 03
2. Chapter: 02 (User Story)---------- 03
3. Chapter: 03a (ER Diagram)---------- 05
4. Chapter: 03b (SQL Queries)---------- 06
5. Chapter: 04 (Screenshots)---------- 09

PharmaCare - Pharmacy Management System

INTRODUCTION

The PharmaCare Pharmacy Management System is a desktop application designed specifically for pharmacies. It helps manage day-to-day pharmacy operations in a more organized and efficient way. It is built using C# with Windows Forms (WinForms) for the interface and SQL Server as the backend database, which ensures secure and reliable data storage.

Purpose:

This system is targeted toward small to medium-sized pharmacies. Its main goal is to automate daily tasks like managing medicines, users, and sales, which reduces human errors and speeds up the workflow. By using C# and SQL Server together, the system is robust, flexible, and easy to maintain

Core Features of the System:

:

1. Inventory Management
   - Allows the pharmacy to add new medicines, update existing products, or remove expired/obsolete items.
   - Helps keep track of stock levels, so pharmacists always know which items need restocking.
2. User Management
   - Enables registration of new users like cashiers, salesmen, or managers.
   - Lets you update user details (name, role, password, contact info).
3. Sales and Billing
   - Facilitates quick processing of sales transactions.
   - Automatically generates bills/invoices for customers, including itemized lists, prices, discounts, and totals.
   - Reduces the need for manual calculations and paperwork.
4. Real-time Tracking
   - Tracks stock levels in real-time, so the system instantly reflects sales or purchases.
   - Enables monitoring of sales data, helping managers identify popular products or slow-moving stock.
5. Reporting
   - Generates basic reports to analyze inventory and sales.

PharmaCare is designed to simplify pharmacy operations, save time, minimize errors, and give management a clear overview of both inventory and sales. Using C# WinForms ensures an easy-to-use interface, while SQL Server ensures all data is secure and retrievable efficiently.

USER STORY

This user story outlines the development of a Pharmacy Management System, aimed at automating the day-to-day operations of a pharmacy. The system ensures efficient management of users, products, and sales. It supports multiple user roles, primarily focusing on **Admin** and **Employee** users, each with specific responsibilities and system access privileges.

Main Entities:

- **Admin:** An Admin has a unique ID, name, email, and password. Admins are responsible for managing the overall system, including product inventory, user creation (both Admins and Employees), and viewing various reports. The system allows multiple admins to be registered and operate concurrently.
- **Employee:** An Employee has a unique ID, name, phone number, email, and password. Employees are responsible for handling the product selling process, searching for products, and generating bills upon successful sales.

Products:

The pharmacy contains multiple types of products, specifically Medicines. Each medicine has specific attributes such as MedicineId (primary key), MName (name), Generic (generic name), Type, Price, and Quantity (stock). Admins can add, update, or remove medicines from the system at any time.

Admin Homepage Features:

The Admin homepage contains several menu options:

- **Dashboard:** View key metrics like sales reports, stock status, and lists of all Admins and Employees.
- **User Management:** o Add User o Update User Info o Remove User **Product Management:** o Add Product o Update Product o Remove Product
- **Logout:** Exit the system.

While adding a new user, the admin must specify the user's role as either **Admin** or **Employee**.

Employee Homepage Features:

The Employee homepage is focused on sales and includes the following menu options:

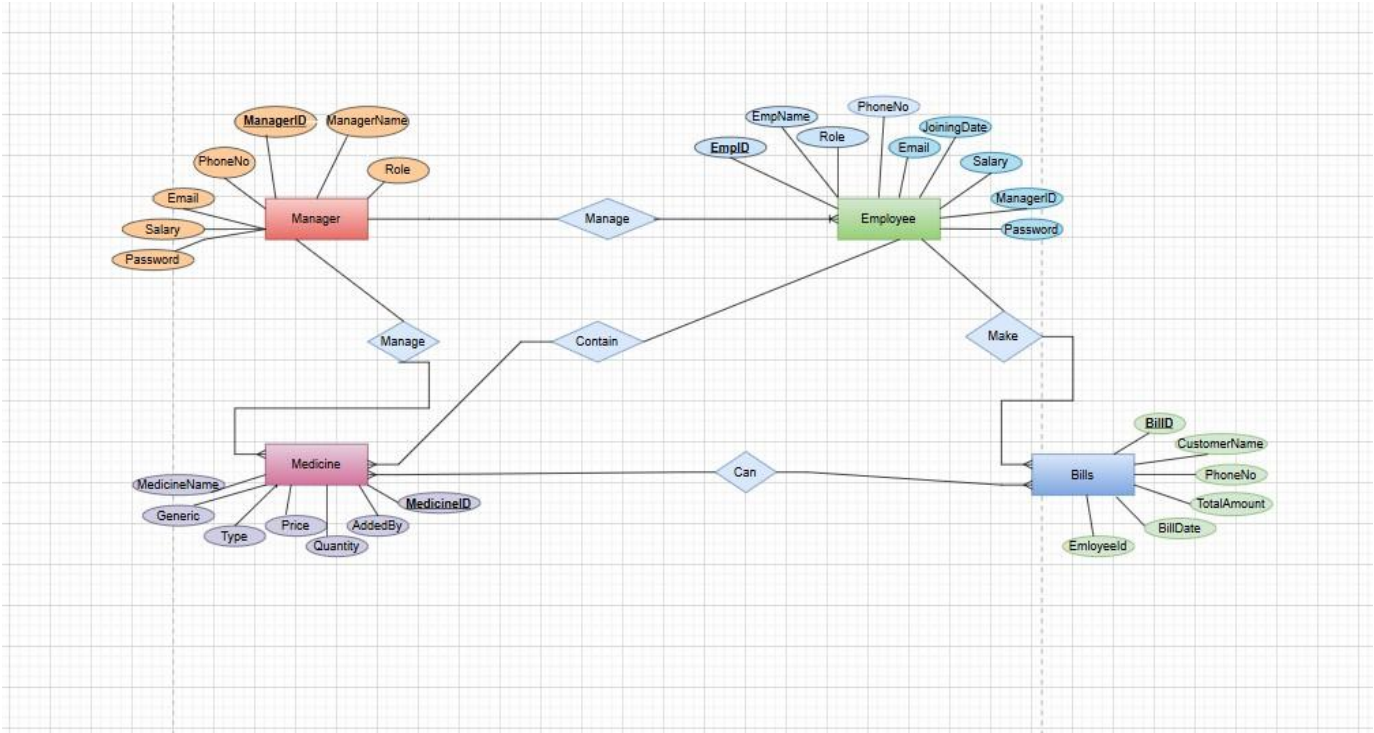
- **Dashboard:** View available products, sales history, and their own profile information.
- **Search Box:** Quickly find products by name, ID, or generic name.
- **Selling:** A feature to add products to a cart and process the sale.
- **Profile:** View personal information.
- **Logout:** Exit the system.

Selling Process:

On the Employee Dashboard, all available products are displayed. The employee can search or browse to select a product and add it to a cart. After selecting products, the employee completes

the transaction, and the system generates a bill. The total price is calculated automatically, and the stock quantity is updated in real time.

ER DIAGRAM



SQL QUERIES

1. Database Creation:

2. Table Creation :

Billa Table

```
CREATE TABLE [dbo].[BillsTb](
```

```
    [BillsId] [nvarchar](50) NOT NULL,
    [CustomerName] [nvarchar](200) NULL,
    [PhoneNo] [nvarchar](30) NULL,
    [TotalAmount] [decimal](14, 2) NULL DEFAULT ((0.00)),
    [BillDate] [datetime] NULL,
    [EmployeeId] [varchar](20) NULL,
PRIMARY KEY CLUSTERED
(
    [BillsId] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
```

Employee table:

```
CREATE TABLE [dbo].[EmployeeTb](
    [EmpId] [varchar](20) NOT NULL,
    [EmpName] [nvarchar](50) NOT NULL,
    [Role] [nvarchar](50) NOT NULL,
    [PhoneNo] [varchar](50) NOT NULL,
    [Email] [nvarchar](50) NOT NULL,
    [JoiningDate] [date] NOT NULL,
    [Salary] [int] NOT NULL,
    [Password] [nvarchar](50) NOT NULL,
 CONSTRAINT [PK_EmployeeTb] PRIMARY KEY CLUSTERED
(
    [EmpId] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
```

Manager Table

```
CREATE TABLE [dbo].[ManagerTb](
    [ManagerId] [varchar](20) NOT NULL,
    [ManagerName] [nvarchar](50) NOT NULL,
    [Role] [nvarchar](30) NOT NULL,
[PhoneNo] [varchar](20) NOT NULL, [Email]
[nvarchar](50) NOT NULL,
    [Salary] [decimal](10, 2) NOT NULL,
    [Password] [nvarchar](50) NULL,
CONSTRAINT [PK_ManagerTb] PRIMARY KEY CLUSTERED
(
    [ManagerId] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
```

Medicine Table

```
CREATE TABLE [dbo].[MedicineTb](
	[MedicineId] [nvarchar](20) NOT NULL,
	[MName] [nvarchar](50) NOT NULL,
	[Generic] [nvarchar](50) NOT NULL,
	[Type] [nvarchar](50) NOT NULL,
	[Price] [decimal](12, 2) NOT NULL,
	[Quantity] [int] NOT NULL,
 CONSTRAINT [PK_MedicineTb] PRIMARY KEY CLUSTERED
(
	[MedicineId] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
```

3. CRUD Operations:

Add:

1. INSERT [dbo].[BillsTb] ([BillsId], [CustomerName], [PhoneNo], [TotalAmount], [BillDate], [EmployeeId]) VALUES (N'B001', N'Ajij', N'0145965000', CAST(1450.00 AS Decimal(14, 2)), CAST(N'2025-09-16 00:00:00.000' AS DateTime), N'E-11')

2. INSERT [dbo].[BillsTb] ([BillsId], [CustomerName], [PhoneNo], [TotalAmount], [BillDate], [EmployeeId]) VALUES (N'B002', N'Rakib', N'01683435462', CAST(1050.00 AS Decimal(14, 2)), CAST(N'2025-09-16 00:00:00.000' AS DateTime), N'E-11')

3. INSERT [dbo].[BillsTb] ([BillsId], [CustomerName], [PhoneNo], [TotalAmount], [BillDate], [EmployeeId]) VALUES (N'B003', N'Purnima', N'018436462', CAST(1050.00 AS Decimal(14, 2)), CAST(N'2025-09-16 00:00:00.000' AS DateTime), N'E-11')

4. INSERT [dbo].[BillsTb] ([BillsId], [CustomerName], [PhoneNo], [TotalAmount], [BillDate], [EmployeeId]) VALUES (N'B004', N'Moitri', N'01488952255', CAST(1450.00 AS Decimal(14, 2)), CAST(N'2025-09-16 00:00:00.000' AS DateTime), N'E-11')

5.INSERT [dbo].[BillsTb] ([BillsId], [CustomerName], [PhoneNo], [TotalAmount], [BillDate], [EmployeeId]) VALUES (N'B005', N'Rahul', N'01488226677', CAST(1250.00 AS Decimal(14, 2)), CAST(N'2025-09-16 00:00:00.000' AS DateTime), N'E-11')

6. INSERT [dbo].[EmployeeTb] ([EmpId], [EmpName], [Role], [PhoneNo], [Email], [JoiningDate], [Salary], [Password]) VALUES (N'E-11', N'Sojib', N'Cashier', N'0198764368', N'cashier@gmail.com', CAST(N'2024-12-17' AS Date), 10000, N'5555')

7. INSERT [dbo].[EmployeeTb] ([EmpId], [EmpName], [Role], [PhoneNo], [Email], [JoiningDate], [Salary], [Password]) VALUES (N'E-12', N'Rabbi', N'Salesman', N'01235789345', N'salesman@gmail.com', CAST(N'2023-04-11' AS Date), 12000, N'6666')

8. INSERT [dbo].[ManagerTb] ([ManagerId], [ManagerName], [Role], [PhoneNo], [Email], [Salary],

[Password]) VALUES (N'M-11', N'Rahim Uddin', N'Manager', N'01567893456', N'rahim@gmail.com', CAST(25000.00 AS Decimal(10, 2)), N'1234')

9. INSERT [dbo].[ManagerTb] ([ManagerId], [ManagerName], [Role], [PhoneNo], [Email], [Salary], [Password]) VALUES (N'M-12', N'Salim', N'Data Security', N'01982343548', N'salim@gmail.com', CAST(30000.00 AS Decimal(10, 2)), NULL)

10. INSERT [dbo].[MedicineTb] ([MedicineId], [MName], [Generic], [Type], [Price], [Quantity]) VALUES (N'1', N'Napa', N'Paracetamol', N'Supplements', CAST(150.00 AS Decimal(12, 2)), 3)

11. INSERT [dbo].[MedicineTb] ([MedicineId], [MName], [Generic], [Type], [Price], [Quantity]) VALUES (N'2', N'Flazil', N'Metformin', N'Antidiabetic', CAST(200.00 AS Decimal(12, 2)), 13)

12. INSERT [dbo].[MedicineTb] ([MedicineId], [MName], [Generic], [Type], [Price], [Quantity]) VALUES (N'3', N'Vidalin Gold', N'Metformin', N'Vitamins', CAST(500.00 AS Decimal(12, 2)), 5)

13. INSERT [dbo].[MedicineTb] ([MedicineId], [MName], [Generic], [Type], [Price], [Quantity]) VALUES (N'4', N'Tuspel', N'Antibiotic', N'Antacid', CAST(400.00 AS Decimal(12, 2)), 13)
14. INSERT [dbo].[MedicineTb] ([MedicineId], [MName], [Generic], [Type], [Price], [Quantity]) VALUES (N'5', N'Sergel', N'Anti Allergic', N'Antacid', CAST(600.00 AS Decimal(12, 2)), 15)

15. INSERT [dbo].[MedicineTb] ([MedicineId], [MName], [Generic], [Type], [Price], [Quantity]) VALUES (N'6', N'Fexo', N'Anti Allergic', N'Supplements', CAST(350.00 AS Decimal(12, 2)), 8)
ALTER TABLE [dbo].[BillsTb] WITH CHECK ADD CONSTRAINT [FK_Bill_Emp] FOREIGN KEY([EmployeeId])

Read (Retrieve)

These SQL statements are used to retrieve and display data from the database, enabling features like searching, reporting, and viewing product lists.

1. @"select* from ManagerTb Where ManagerId='";

2. @"SELECT * FROM EmployeeTb WHERE LOWER(EmpId) = LOWER('" + this.Ussr.Trim() + "')"

3. "select * from MedicineTb;"

4. "select BillsId from BillsTb order by BillsId desc;"

5. @"select * from ManagerTb where ManagerId = '" + userId + "' and Password = '" ;

6. "select * from EmployeeTb where EmpId = '" + userId + "' and Password = '" + password ;
7. "select * from ManagerTb;"

8. "select ManagerId from ManagerTb order by ManagerId desc;

9. @"select* from ManagerTb Where ManagerId='" + Search + "'or ManagerName='" + Search + "' PhoneNo='" +Search + "';

10. "select * from EmployeeTb;

11. "select EmpId from EmployeeTb order by EmpId desc;

12. @"SELECT * FROM EmployeeTb WHERE EmpId LIKE '" +serach + @"%' OR EmpName LIKE '" + Search + @"%';

13. "select MedicineId from MedicineTb order by MedicineId desc;"

14. @"select* from MedicineTb Where MedicineId like'" + Search.+ "%'or MName like'" + Search + "%'or Generic like'" + Search + "%';";

15. "select * from ManagerTb where ManagerId ='" + this.UserId + "';"

16. string sql = @"SELECT * FROM BillsTb WHERE BillsId LIKE @search OR CustomerName LIKE @search OR PhoneNo LIKE @search";

Update

These commands are used to modify existing data in the database, such as updating a user's password or changing a product's price and stock quantity.

1. @"UPDATE ManagerTb SET ManagerName = @ManagerName, Role = @Role, PhoneNo = @PhoneNo,Email = @Email,Salary = @Salary, Password = @Password WHERE ManagerId = @Id;";

2. string sql = @"UPDATE MedicineTb SET Quantity = @Quantity WHERE MedicineId = @MedicineId;";

3. @"UPDATE ManagerTb SET ManagerName = @ManagerName, Role = @Role PhoneNo = @PhoneNo, Email = @Email,Salary = @Salary, Password = @Password WHERE ManagerId = @ManagerId;"

4. string sql = @"UPDATE EmployeeTb SET EmpName = @EmpName, Role = @Role, PhoneNo = @PhoneNo, Email = @Email, JoiningDate = @JoiningDate, Salary = @Salary, Password = @Password WHERE EmpId = @EmpId";

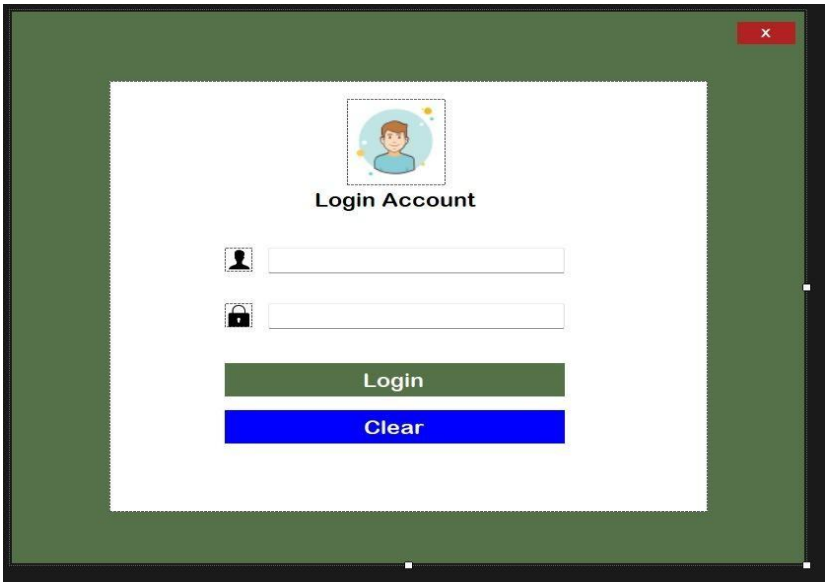
5. UPDATE MedicineTb SET MName = @mname, Generic = @generic, Type = @type, Price = @price, Quantity = @quantity WHERE MedicineId = @id;

Delete

These commands are used to permanently remove records from the database.

1. DELETE FROM BillsTb WHERE BillsId = @id;
2. DELETE FROM EmployeeTb WHERE EmpId = @id;
3. DELETE FROM ManagerTb WHERE ManagerId = @id;
4. DELETE FROM MedicineTb WHERE MedicineId = @id;

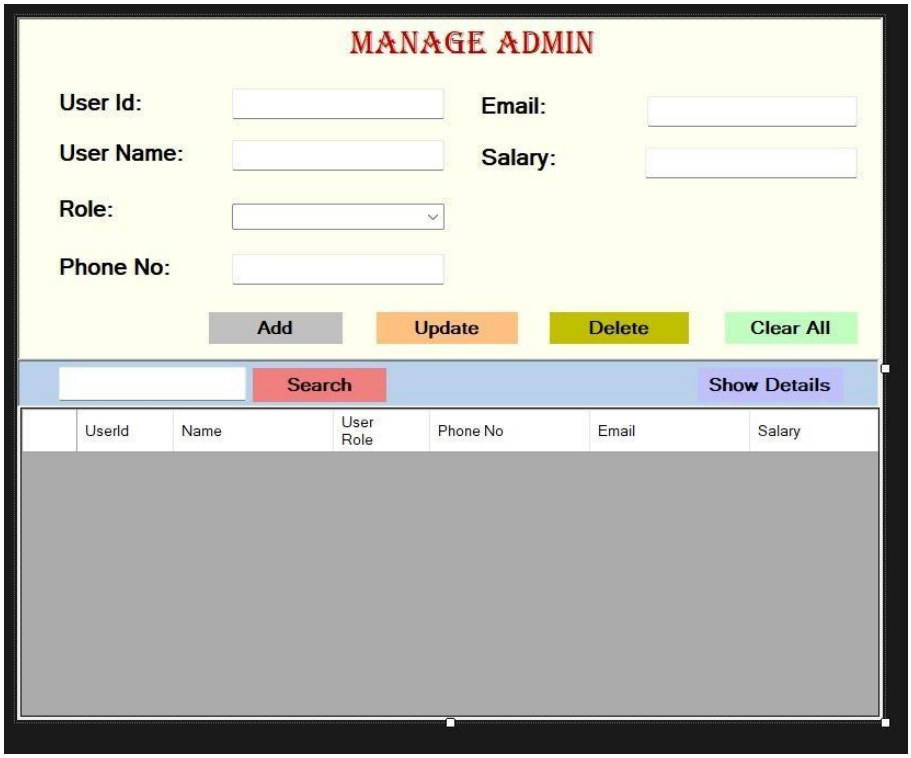
3. Screenshots



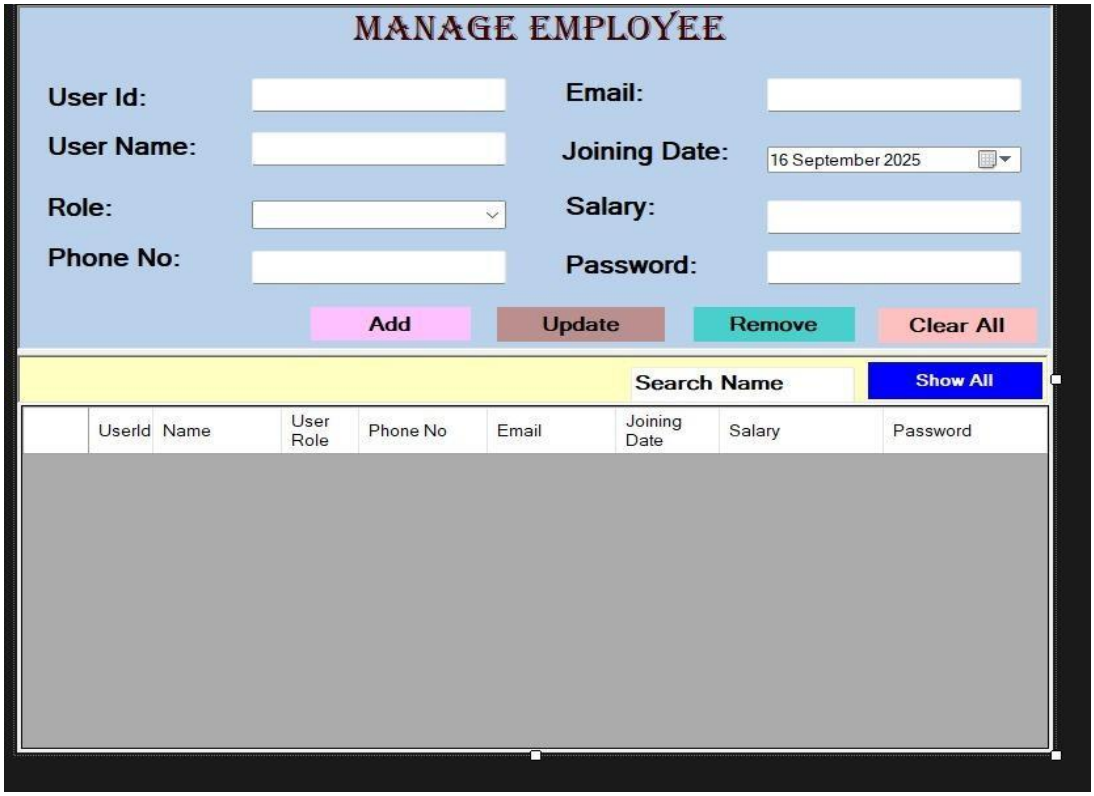
Login Form



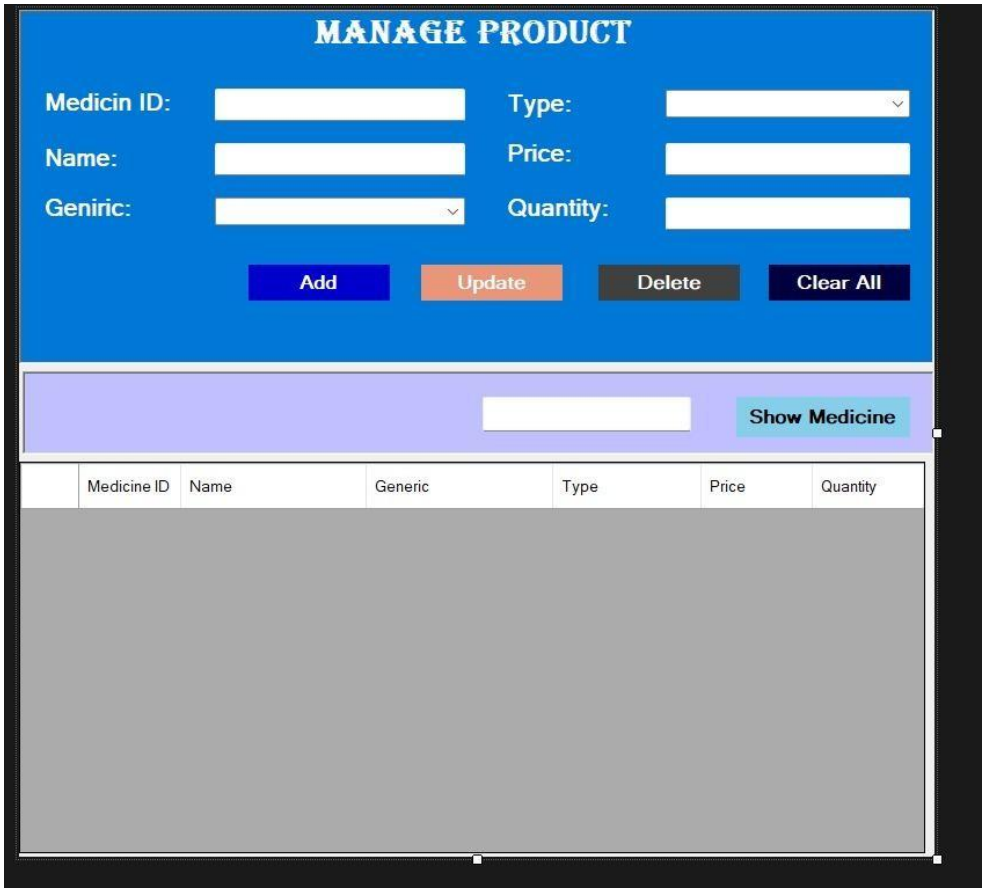
Admin Form



Admin Manage Form



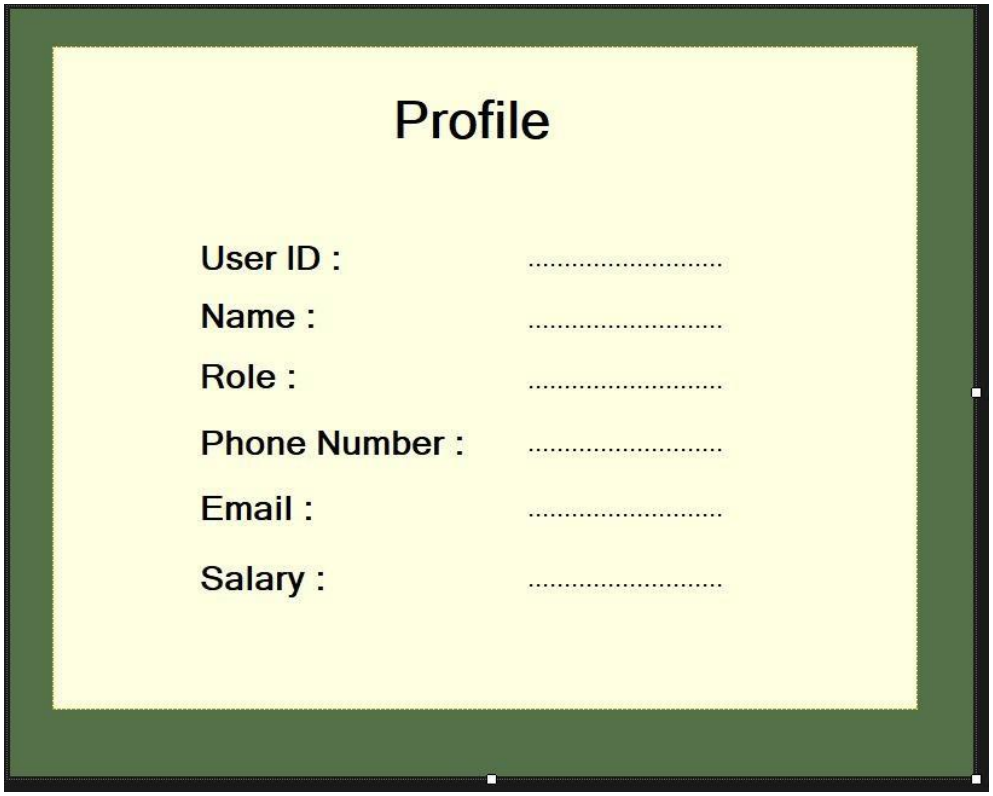
Employee Manage Form



Product Manage Form



Sell Information Form



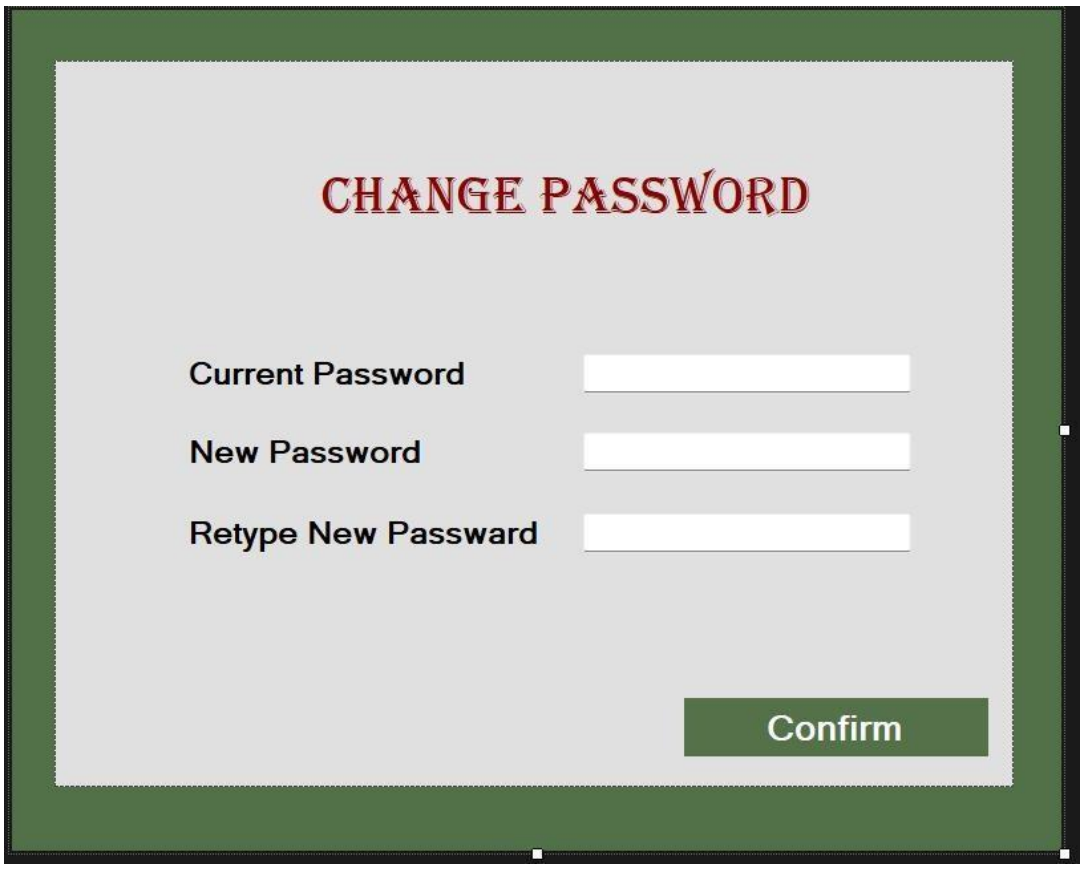
User Form



Employee Form



Employee Information Form



Admin Password Form



Employee Password Form



Product List Form



Bill Information Form